

Week 3 Technical Report

Oracle Container Engine for Kubernetes built with reusable Terraform modules

VCN-native pod networking · managed node pool · block-volume persistence · load-balancer exposure

Field	Value
Author	Andrew Hany
Programme	Ejada Summer Internship 2026 — Cloud Build track
Mentor	Belal Mokhtar
Week	Week 3 — DevOps / Kubernetes and Terraform modules
Tenancy	ociejada
Compartment	intern-04-andrew-hany-cmp
Region	me-jeddah-1 (Jeddah)
Assigned	Monday 17 August 2026
Submitted	_____

Contents

Contents.....	2
1. Introduction	4
1.1 Assignment	4
1.2 How that shaped the deliverable.....	4
1.3 Objectives	4
1.4 Relationship to Weeks 1 and 2	4
2. Background	5
2.1 Managed Kubernetes on OCI.....	5
2.2 Pod networking: VCN-native versus overlay.....	5
2.3 Persistent storage for containers.....	6
3. Architecture	7
3.1 Components.....	7
3.2 Address plan	7
3.3 Design decisions.....	8
Public API endpoint.....	8
Private worker nodes	8
Security lists rather than NSGs	8
Managed node pool rather than virtual nodes.....	9
One shared log group	9
3.4 Traffic paths	9
3.5 What Terraform does not own	9
The boundary also runs the other way	9
4. Terraform module design	11
4.1 Why modules	11
4.2 The Subnet module.....	11
4.3 The OKE module.....	11
4.4 Dynamic blocks	12
Nesting	12
The zero-or-one idiom	13
When not to use one	13
4.5 Conditional expressions.....	13
The ternary — conditional values.....	13
count — conditional resources.....	14
The version of that block which did not work	14
for_each — conditional and repeated at once	14
try, coalesce, compact — resolving optional values.....	15
optional() — conditional inputs	15

validation and precondition — failing fast, in your own words.....	15
4.6 Reusability in practice	15
Deciding what stays in the root	15
One knob, not two	16
Names derived from one input.....	16
Outputs are part of the contract	16
Version constraints	16
4.7 What the abstraction bought.....	16
4.8 The reusability test, and its result.....	17
5. The Kubernetes workload	18
5.1 The persistent volume claim.....	18
5.2 The deployment.....	18
5.3 The service and load balancer	18
6. Implementation	20
6.1 Method	20
6.2 Pre-flight	20
6.3 Automation and evidence capture	21
6.4 Problems encountered	22
7. Verification.....	25
7.1 Summary.....	25
7.2 VCN-native pod networking.....	25
7.3 Managed worker node pool	26
7.4 External block volume.....	28
7.5 Load balancer exposure.....	29
7.6 The persistence test.....	31
7.7 Flow logs	31
8. Security review.....	34
9. Cost and teardown.....	34
10. Conclusion.....	35
10.1 What I would do differently.....	35
Appendix A — Repository contents	37
Appendix B — Command reference	37
Appendix C — References.....	37
Appendix D — Console evidence: the network layer	39

1. Introduction

1.1 Assignment

Week 3 of the Cloud Build track moves from single virtual machines to container orchestration. The brief set two pieces of work that depend on each other.

The Terraform piece introduces modules — specifically a Subnet module bundling a subnet with its route table, its security list and its flow logging, and an OKE module for the cluster and everything the cluster needs. The instruction attached to it was explicit: make the modules generic and reusable, avoid hardcoding values inside module code, and pass values through variables adapted from the root configuration. Three concepts were named for research: dynamic blocks, conditional expressions, and what makes a module reusable and configurable.

The lab piece exercises those modules: create an OKE cluster with VCN-native pod networking and a managed worker node pool, deploy an application to it, attach an external block volume, and expose the application through a load balancer.

The brief closed with the sentence that determined how this work was prioritised: *“The goal is not only to make it work, but to build it in a clean, reusable, and configurable Terraform structure.”*

1.2 How that shaped the deliverable

Read literally, that sentence says the module design is the deliverable and the running cluster is the evidence for it. This report is organised the same way. Section 4 — the module design — is the longest section and the one to read first; the lab itself is treated as the demonstration that the modules produce working infrastructure rather than merely valid HCL.

A concrete consequence: no branch anywhere inside either module exists to serve one specific subnet or one specific cluster in this lab. Every value that varies lives in the root configuration or in `terraform.tfvars`. Section 4.8 states the test used to check that claim and reports the result.

1.3 Objectives

- Build a Subnet module owning four resources — subnet, route table, security list, and VCN flow logs — with no lab-specific values inside it.
- Build an OKE module owning the cluster, its node pools and its add-ons, supporting more configurations than this lab needs.
- Demonstrate dynamic blocks, conditional expressions and the reusability rules, and explain each in terms of the problem it solves.
- Stand up an OKE cluster using VCN-native pod networking with a managed worker node pool.
- Deploy an application whose data lives on an external OCI Block Volume, not in the container image.
- Expose that application to the internet through an OCI Load Balancer provisioned by Kubernetes.
- Prove each of the four lab requirements with evidence rather than assertion.

1.4 Relationship to Weeks 1 and 2

Week 1 built a VCN, a compute instance and storage, first through the Console and then through Terraform. Week 2 added a load balancer in a public subnet in front of a private compute instance, with the application files on OCI File Storage over NFS, and was written entirely in Terraform in a single flat root module.

Week 3 changes two things. The workload becomes orchestrated rather than a single instance, and the Terraform stops being flat. Week 2's `network.tf` declared each subnet, route table and security list as separate top-level resources; that file was readable at four subnets and would not have been at ten. The Subnet module is the direct answer to that.

One inherited decision is worth flagging early, because it looks inconsistent and is not. Week 2 used Network Security Groups; Week 3 uses security lists. Section 3.3 explains why the choice went the other way here.

2. Background

2.1 Managed Kubernetes on OCI

Oracle Container Engine for Kubernetes runs the control plane — API server, scheduler, controller manager and etcd — as an Oracle-managed service. The tenancy provides the worker nodes and the network they sit in. A managed node pool means OCI creates and lifecycles the worker compute instances from a declared shape, size and image, rather than the instances being declared individually.

Two OCI controllers run inside every OKE cluster and matter to this design because they create real infrastructure that Terraform does not manage:

- The cloud-controller-manager watches Services of type LoadBalancer and provisions an OCI Load Balancer for each one.
- The Block Volume CSI driver watches PersistentVolumeClaims and provisions, attaches and formats OCI Block Volumes.

Section 3.5 returns to this boundary, because it has consequences for both teardown and plan cleanliness.

2.2 Pod networking: VCN-native versus overlay

OKE offers two CNI choices, and the brief specified the first.

	OCI_VCN_IP_NATIVE	FLANNEL_OVERLAY
Pod address	A real VCN IP from a dedicated pod subnet	An overlay address from a private range
Routable in the VCN	Yes, directly	No, encapsulated between nodes
Visible to security lists / NSGs	Yes — a pod is a first-class VCN endpoint	No — only node-to-node traffic is visible
Appears in VCN flow logs	Yes	No, only the tunnel
Address consumption	One VCN address per pod	One VCN address per node
Sizing consequence	The pod subnet must be large	A small worker subnet suffices

The trade is directness against address space. Pods that hold real VCN addresses can be filtered, logged and reached like any other resource in the network — and that is precisely why the pod subnet in this design is a /19 while every other subnet is a /24 or smaller. Section 3.2 works through the arithmetic.

2.3 Persistent storage for containers

A container filesystem is ephemeral: anything written into it disappears when the container is replaced. Kubernetes solves this with a PersistentVolumeClaim, a request for storage that a driver satisfies by provisioning a real volume.

On OKE the relevant storage class is `oci-bv`, backed by the provisioner `blockvolume.csi.oraclecloud.com`. Three of its properties shape the manifests in section 5:

- Minimum size is 50 GB. Smaller requests are rounded up rather than rejected.
- Access mode is `ReadWriteOnce` — a block volume attaches to one node at a time.
- `volumeBindingMode` is `WaitForFirstConsumer`, so the volume is not created until a pod has been scheduled and the availability domain is known.

The contrast with Week 2 is instructive. Week 2 used OCI File Storage over NFS, which is `ReadWriteMany` and can be mounted by many instances at once. Week 3 uses block storage, which cannot. That single difference is what forces the Deployment in section 5.2 to one replica with a `Recreate` strategy — it is a property of the storage primitive, not a limitation of the application.

3. Architecture

Week 3 — OKE Cluster with VCN-Native Pod Networking

Ejada Summer Internship 2026 · Cloud Build track · Andrew Hany · region me-jeddah-1 · compartment intern-04-andrew-hany-cmp

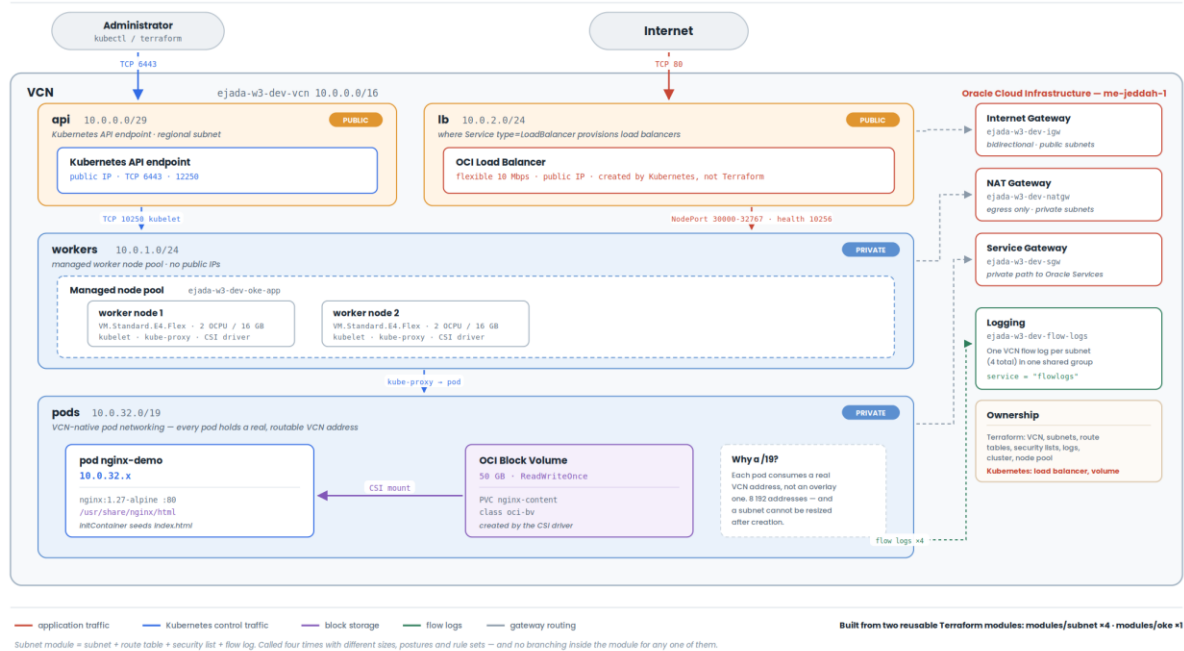


Figure 1. Week 3 target architecture. Four subnets, three gateways, an OKE cluster with VCN-native pod networking, a block volume attached to the application pod, and a public load balancer.

3.1 Components

Component	Role
VCN 10.0.0.0/16	Network boundary for the whole stack.
api subnet (public)	Hosts the Kubernetes API endpoint. Public so kubectl works without a bastion.
workers subnet (private)	Managed worker nodes. No public IPs; outbound only, through NAT.
pods subnet (private)	Supplies VCN-native pod addresses.
lb subnet (public)	Where Kubernetes provisions load balancers for Services.
Internet gateway	Bidirectional internet path for the two public subnets.
NAT gateway	Outbound-only internet path for the two private subnets.
Service gateway	Private path to the Oracle Services Network for OKE, OCIR and Object Storage.
Log group + 4 flow logs	One VCN flow log per subnet, all writing into one shared log group.
OKE cluster	Enhanced cluster, VCN-native CNI, public API endpoint.
Managed node pool	Two VM.Standard.E4.Flex nodes, 2 OCPU / 16 GB each.
Block volume (50 GB)	Created by the CSI driver from the PVC. Holds the served content.
Load balancer	Created by the cloud-controller-manager from the Service.

3.2 Address plan

Sizing follows Oracle's reference configuration for an OKE cluster using VCN-native pod networking with a public API endpoint, private workers and public load balancers.

Subnet	CIDR	Addresses	Access	Purpose
api	10.0.0.0/29	8	Public	Kubernetes API endpoint
workers	10.0.1.0/24	256	Private	Managed worker nodes
lb	10.0.2.0/24	256	Public	Service type=LoadBalancer
pods	10.0.32.0/19	8 192	Private	VCN-native pod addresses

The /19 deserves justification, because at first sight it is wildly oversized. Two nodes at the configured maximum of 31 pods each is 62 addresses — a /26 would do today.

Two facts make that reasoning wrong. First, with VCN-native networking every pod permanently consumes a VCN address, so consumption scales with pods rather than with nodes. Second, and decisively, an OCI subnet cannot be resized after creation. Growing past a /24 pod subnet would mean creating a new subnet, a new node pool, and draining the cluster onto it. The pod subnet is therefore sized for the cluster this might become, not the one it starts as; the addresses cost nothing while unused, and the migration would cost a great deal.

services_cidr is 10.96.0.0/16, deliberately outside the VCN range. ClusterIP addresses are virtual — they exist only inside kube-proxy's rules and never appear on the wire — so they must not collide with anything routable.

3.3 Design decisions

Public API endpoint

The endpoint gets a public IP so that kubectl works from a laptop with no bastion and no VPN. This is a lab convenience and is treated as one: the source range is a variable, kubectl_source_cidr, which defaults to 0.0.0.0/0 and exists so that narrowing it to a single /32 is a one-line change. Section 8 lists it as the first thing to change in any non-lab deployment.

Private worker nodes

Worker nodes have no public IPs at all. Their only inbound paths are the control plane on TCP 10250 and the load balancer on the NodePort range. Everything outbound — image pulls, OS packages — goes through the NAT gateway, and OCI service traffic goes through the service gateway without touching the internet.

Security lists rather than NSGs

Week 2 used Network Security Groups; this design uses security lists, and the reversal is deliberate rather than inconsistent.

The two tools solve different shapes of problem. An NSG attaches to a VNIC, so it expresses "these particular resources have this posture" — which is what Week 2 needed, where a load balancer, an application instance and a File Storage mount target with three different postures shared subnets. A security list attaches to a subnet, so it expresses "everything in this subnet has this posture".

In Week 3 the four subnets each contain exactly one kind of thing, and the security boundary lines up precisely with the subnet boundary. That is the case security lists were designed for, and it also makes

the Subnet module a coherent unit: a subnet and its filtering genuinely are one thing here, which is what makes bundling them into one module honest rather than arbitrary.

Managed node pool rather than virtual nodes

The brief asked for a managed node pool. It is also the choice the rest of the lab requires: virtual nodes are serverless and do not support block volume persistent claims the same way, so the storage requirement and the node pool requirement are connected.

One shared log group

The Subnet module can create its own log group when none is supplied. The root does supply one, so all four flow logs land in a single group and one Log Search query covers the whole VCN. Section 4.5 shows the conditional that makes both behaviours possible without a flag to explain.

3.4 Traffic paths

```

Inbound to the application
  client -> OCI Load Balancer      (public IP, lb subnet)
          -> worker node NodePort (30000-32767, workers subnet)
          -> kube-proxy
          -> pod 10.0.32.x         (pods subnet)
          -> nginx :80 -> file read from the block volume

Administration
  laptop -> public API endpoint :6443 (api subnet) -> Oracle-managed control plane

Control plane to workload
  api subnet -> workers subnet  TCP (kubelet on 10250, webhooks, aggregated APIs)
  api subnet -> pods subnet     all (exec, logs, port-forward)

Outbound
  workers/pods -> NAT gateway     container registries, OS packages
  workers/pods -> service gateway OKE, OCIR, Object Storage - stays on Oracle's network

```

3.5 What Terraform does not own

Two resources in the running system are created by Kubernetes controllers rather than by Terraform, and both consequences of that are worth stating explicitly because both surface late.

Resource	Created by	Consequence
OCI Load Balancer	cloud-controller-manager, from the Service	Invisible to terraform destroy; its VNIC blocks deletion of the lb subnet.
50 GB Block Volume	CSI driver, from the PersistentVolumeClaim	Same — orphans silently and keeps billing.

This is why teardown runs `kubectl delete` first and `terraform destroy` second, and why that ordering is written into both the runbook and the automation script rather than left to memory.

The boundary also runs the other way

The cloud-controller-manager does not stop at creating the load balancer. Its default setting, `oci-load-balancer-security-list-management-mode: "All"`, also makes it write rules into the security lists attached to the load balancer subnet and the worker subnet — the two lists this project's Terraform owns and generates from its own rule definitions.

The result would be two controllers editing one object. Terraform's next plan would show the controller's rules being removed; apply would remove them; the controller would put them back. Nothing breaks, because Terraform already opens the same ports the controller wants — but the plan is never clean again, which is corrosive on a codebase whose entire claim is that the plan is the review artefact.

The Service therefore sets the mode explicitly:

```
service.beta.kubernetes.io/oci-load-balancer-security-list-management-mode: "None"
```

k8s/30-service-lb.yaml

Terraform declares the network; Kubernetes declares the workload; neither edits the other's objects. This is called out because the default is the setting that causes the problem, and the symptom appears days after the cause.

4. Terraform module design

This is the primary deliverable. Sections 4.1 to 4.3 describe what was built, 4.4 to 4.6 cover the three researched concepts through the code that uses them, and 4.7 and 4.8 assess the result.

4.1 Why modules

Week 2's configuration was flat: every subnet, route table and security list was a separate top-level resource in one file. It worked, and it was already near the edge of readable at four subnets. Adding four more would have meant copying twelve resource blocks and editing each one — the classic setting for a copy-paste error that produces valid HCL and a wrong network.

A module answers this by naming a repeating structure once. The governing rule for the whole design is one sentence:

A module owns structure. The root owns values.

Every other rule applied here follows from it. It also gives a mechanical test for "did I hardcode something": search the module for a CIDR, an OCID, a shape name, a region or an environment name. Any hit is a defect.

4.2 The Subnet module

modules/subnet owns a subnet and the three things that always travel with it.

#	Resource	Conditional on
1	oci_core_subnet	always created
2	oci_core_route_table	var.create_route_table (default true)
3	oci_core_security_list	var.create_security_list (default true)
4	oci_logging_log (VCN flow logs)	var.flow_logs_enabled (default true)
4a	oci_logging_log_group	var.create_flow_logs_log_group (default true)

It is called four times. The four calls differ in prefix length (from /29 to /19), in public versus private posture, in routing (internet gateway versus NAT plus service gateway), and in rule count — from three rules on the load balancer subnet to thirteen on the worker subnet. Counted from the running VCN: load balancer 1 ingress / 2 egress, pods 3 / 6, API endpoint 6 / 5, workers 6 / 7. The module code is identical for all four.

Nothing in the module refers to Kubernetes, to 10.0.x, or to Ejada. Read on its own it is "an OCI subnet with its routing, its filtering and its logging" — which is the property that makes it liftable into an unrelated project.

4.3 The OKE module

modules/oke owns the cluster, its node pools and its add-ons. It supports meaningfully more than this lab uses: either CNI type, public or private API endpoints, any number of node pools through for_each over a map, and optional cluster add-ons.

Two things it deliberately does not hardcode are the Kubernetes version and the node image. Both are discovered:

```
data "oci_containerengine_cluster_option" "this" { cluster_option_id = "all" ... }
data "oci_containerengine_node_pool_option" "this" { node_pool_option_id = "all" ... }

locals {
  available_versions = data.oci_containerengine_cluster_option.this.kubernetes_versions
  cluster_version    = coalesce(
    var.kubernetes_version,
    element(local.available_versions, length(local.available_versions) - 1),
  )
}
```

modules/oke/main.tf — caller override first, discovery as fallback

The image lookup goes further, filtering OKE's published sources by the resolved version and by CPU architecture inferred from the shape name, and excluding GPU builds. An Ampere node pool automatically receives an aarch64 image without the caller having to know that architecture-specific images exist.

One output is deliberately absent. The cluster kubeconfig is *not* emitted as a Terraform output, because doing so would write a working cluster credential in plain text into terraform.tfstate. The module outputs the `oci ce cluster create-kubeconfig` command instead.

4.4 Dynamic blocks

Terraform separates arguments, whose values can be computed, from blocks, whose structure is normally fixed when the file is written. A security list with nine ingress rules therefore means nine hand-written blocks, and a module serving four subnets with four different rule sets cannot be written that way at all.

A dynamic block generates a repeatable block from a collection:

```
dynamic "route_rules" {
  for_each = var.route_rules
  content {
    destination      = route_rules.value.destination
    destination_type = route_rules.value.destination_type
    network_entity_id = route_rules.value.network_entity_id
    description      = route_rules.value.description
  }
}
```

modules/subnet/main.tf

Nesting

The pattern doing most of the work is a dynamic block whose content contains another. The security list nests three levels deep — rule, then protocol options, then the source port range:

```
dynamic "ingress_security_rules" {
  for_each = var.ingress_rules
  content {
    protocol = ingress_security_rules.value.protocol
    source   = ingress_security_rules.value.source

    dynamic "tcp_options" {
      for_each = ingress_security_rules.value.tcp_options == null ? [] :
[ingress_security_rules.value.tcp_options]
      content {
        min = tcp_options.value.min
        max = tcp_options.value.max
      }
    }
  }
}
```

```

        dynamic "source_port_range" {
            for_each = tcp_options.value.source_port_range == null ? [] :
[tcp_options.value.source_port_range]
            content { min = source_port_range.value.min
                    max = source_port_range.value.max }
        }
    }
}
}
}

```

modules/subnet/main.tf — abbreviated; udp_options and icmp_options follow the same shape

The zero-or-one idiom

tcp_options is not a repeatable block — a rule has at most one. So why generate it dynamically? Because there is no other way to make a single block conditional. for_each over a list of length zero or one emits the block never or once: an empty list means the block is absent, a one-element list means it is present.

It reads oddly the first time and then becomes the standard way to express "include this block only if".

The same idiom appears twice more in the OKE module, both times for a genuine either-or in the OCI API:

```

# Flex shapes require an OCPU/memory split; fixed shapes must not have one.
dynamic "node_shape_config" {
    for_each = each.value.ocpus == null && each.value.memory_in_gbs == null ? [] : [1]
    content { ocpus = each.value.ocpus, memory_in_gbs = each.value.memory_in_gbs }
}

# Pod network details apply only to VCN-native pod networking.
dynamic "node_pool_pod_network_option_details" {
    for_each = local.is_vcn_native ? [1] : []
    content {
        cni_type          = var.cni_type
        pod_subnet_ids    = each.value.pod_subnet_ids
        max_pods_per_node = each.value.max_pods_per_node
    }
}

```

modules/oke/main.tf

The literal [1] is a presence flag: content never reads .value, so the element's value is irrelevant.

When not to use one

Dynamic blocks hide structure. A reader can no longer see what a resource contains by reading the file — they have to trace the input. That cost is worth paying inside a module whose purpose is configurability, and is not worth paying in a root module for a fixed, known set of blocks, where three literal blocks are clearer than a dynamic over a three-element local. They also cannot be driven by values that are unknown at plan time, since Terraform must expand the blocks before it can produce a plan.

4.5 Conditional expressions

Terraform has no if statement. It has six mechanisms that together do the same work, and this project uses all six.

The ternary — conditional values

```

locals { prohibit_public_ip = !var.is_public }

pods_cidr = local.is_vcn_native ? null : var.pods_cidr

```

Both branches are checked for type compatibility, so they must agree in type. `null` is compatible with everything, which makes condition `? value : null` the standard way to set an attribute only sometimes — used above to send `pods_cidr` only when the CNI is actually an overlay.

count — conditional resources

```
resource "oci_core_route_table" "this" {
  count = var.create_route_table ? 1 : 0
}

resource "oci_logging_log_group" "this" {
  count = var.flow_logs_enabled && var.create_flow_logs_log_group ? 1 : 0
}
```

The log group gives the module two behaviours: a caller who supplies a group gets all its flow logs written into it — which is what the root does, so one Log Search covers the whole VCN — and a caller who supplies nothing gets a group created for it.

The version of that block which did not work

This is worth recording in full, because the first version was more elegant and could not work, and the reason is a genuine boundary in how Terraform operates rather than a slip.

The module originally inferred the caller's intent from the value itself, so there was no extra flag to document:

```
count = var.flow_logs_enabled && var.flow_logs_log_group_id == null ? 1 : 0 # WRONG
```

``terraform validate`` passed. ``terraform plan`` failed with "Invalid count argument — the count value depends on resource attributes that cannot be determined until apply."

The root passes ``oci_logging_log_group.flow_logs[0].id``, the OCID of a log group being created in the same run. At plan time that value is unknown. Comparing an unknown to `null` does not give false — it gives an **unknown boolean**. And ``count`` has to be resolved during planning, because Terraform must know how many instances of a resource exist before it can produce a plan at all. An unknown count is not a value Terraform can work with.

The fix is one more input, ``create_flow_logs_log_group``, which asks the caller to state the intent rather than inferring it from a value that may not exist yet. That also makes the log group consistent with the two conditional resources the module already had — ``create_route_table`` / ``route_table_id`` and ``create_security_list`` / ``security_list_ids`` both pair a boolean with an optional OCID. What looked like an elegant special case turned out to be the odd one out.

Two things worth taking from it. First, the rule: never derive ``count`` or ``for_each`` from anything that traces back to another resource's attributes — pass a flag, or use a value known before apply. Second, and more useful: ``validate`` could not have caught this. Validation checks syntax, types and references without evaluating anything that depends on real infrastructure, so unknown-ness does not exist yet at that stage. A configuration can be entirely valid and still be unplannable. ``plan`` is the first stage that tells the truth, which is a good argument for running it early even when nothing is going to be applied.

for_each — conditional and repeated at once

```
resource "oci_containerengine_addon" "this" {
  for_each = var.cluster_type == "ENHANCED_CLUSTER" ? var.cluster_addons : {}
}
```

```
}
```

An empty map creates nothing, so one expression covers zero, one or many. And unlike count, the instances get stable string keys: removing one node pool from the map does not renumber and destroy the others. The rule applied throughout is `for_each` over count whenever the collection can change.

try, coalesce, compact — resolving optional values

```
# "Caller's value, else the one we created, else nothing."
flow_log_group_id = var.flow_logs_enabled ? coalesce(
  var.flow_logs_log_group_id,
  try(oci_logging_log_group.this[0].id, null),
) : null
```

optional() — conditional inputs

Since Terraform 1.3, object type constraints can mark attributes optional and give them defaults. This is what lets a caller write these two elements in the same list:

```
{ protocol = "all", source = "10.0.32.0/19", description = "Pod to pod" }
{ protocol = "6", source = "0.0.0.0/0", tcp_options = { min = 6443, max = 6443 } }
```

Terraform normalises both to the full object shape, filling the gaps. Without `optional()`, every element would have to spell out every attribute including a pile of nulls, and the rule definitions in `locals.tf` would be roughly four times longer and unreadable — which in a file full of security rules is not a cosmetic concern.

validation and precondition — failing fast, in your own words

```
variable "cidr_block" {
  validation {
    condition     = can(cidrhost(var.cidr_block, 0))
    error_message = "cidr_block must be a valid IPv4 CIDR, e.g. 10.0.1.0/24."
  }
}

lifecycle {
  precondition {
    condition = !local.is_vcn_native || alltrue([
      for k, np in var.node_pools : length(np.pod_subnet_ids) > 0
    ])
    error_message = "cni_type = OCI_VCN_IP_NATIVE requires every node pool to set pod_subnet_ids."
  }
}
```

validation checks one variable in isolation; precondition can check across variables and resources, which validation cannot. The `!A || B` construction is how "if A then B" is written without an if.

The practical value is the difference between a clear failure in two seconds and a confusing OCI API error twenty minutes into an apply that has already created a cluster.

4.6 Reusability in practice

Deciding what stays in the root

The Subnet module does not create the VCN, the internet gateway, the NAT gateway or the service gateway. Those are VCN-scoped and shared, and would otherwise be created four times over.

The test used: if two callers of this module in the same stack would each need it, and there should only be one, it belongs in the root. The module never learns that a NAT gateway exists — it takes a route rule carrying a `network_entity_id` and renders it.

One knob, not two

```
variable "is_public" { type = bool, default = false }

locals {
  prohibit_public_ip      = !var.is_public
  prohibit_internet_ingr = !var.is_public
}
```

The OCI API exposes two negatively-named booleans. The module exposes one positively-named one, because callers think in "public or private", not in double negatives. Exposing intent rather than implementation is most of what makes a module pleasant to call.

Names derived from one input

Every resource takes its display name from `var.name`, so one variable renames the whole set consistently. The root supplies `"${var.prefix}-workers"`, which means `var.prefix` renames the entire stack.

Outputs are part of the contract

Outputs include inputs echoed back — the subnet's CIDR, for instance — so a caller can build downstream configuration from module outputs rather than repeating literals it already passed in.

Version constraints

The modules declare `required_providers` with a loose `">= 5.0.0"`; the root pins `"~> 8.0"`. This asymmetry is intentional: a module should state its minimum and let whoever consumes it choose the exact version, while the root is where reproducibility is decided. For the same reason `.terraform.lock.hcl` is committed rather than ignored — the lock file exists to be committed.

`required_version` is `">= 1.3.0"` and is not decorative: `optional()` in object type constraints arrived in 1.3, and this code does not work below it.

4.7 What the abstraction bought

The clearest measure is the security list. Written by hand, the worker subnet's rules are roughly forty lines of repetitive HCL, and the other three subnets are three more slabs of the same shape. Generated, the module contains one dynamic block and the root contains this:

```
workers_ingress = [
  { protocol = "all", source = local.cidr_workers, description = "Worker to worker" },
  { protocol = "6",   source = local.cidr_api,   tcp_options = { min = 10250, max = 10250 } },
],
{ protocol = "all", source = local.cidr_pods, description = "Pods -> workers" },
{ protocol = "6",   source = local.cidr_lb,   tcp_options = { min = 30000, max = 32767 } },
},
{ protocol = "6",   source = local.cidr_lb,   tcp_options = { min = 10256, max = 10256 } },
},
local.icmp_pmtu_ingress,
]
```

terraform/locals.tf

The gain is not only brevity. Because every rule is expressed in terms of `local.cidr_*` rather than literal addresses, changing the address plan in `terraform.tfvars` re-wires every rule in the cluster with no edit to `locals.tf` and none inside the modules. The load balancer subnet's rules go further and are generated from a for expression over `var.app_ingress_ports`, so adding a port to `terraform.tfvars` adds a rule.

4.8 The reusability test, and its result

The question asked of each module was: could this folder be dropped into an unrelated project, unchanged, and used?

- `modules/subnet` — yes. It contains no reference to Kubernetes, to this address plan, or to this organisation. It is a general-purpose OCI subnet with routing, filtering and logging.
- `modules/oke` — yes. It takes subnet OCIDs and a node pool map, and serves a flannel cluster with a private endpoint and three node pools as readily as this lab's single VCN-native pool.

The evidence that the abstraction actually holds is `subnets.tf`: four subnets, spanning public and private, /29 to /19, three rules to thirteen, produced by four calls to one module — with no branch inside that module written for any particular one of them.

5. The Kubernetes workload

Four manifests, applied in filename order.

File	Resource	Demonstrates
00-namespace.yaml	Namespace ejada-w3	Isolation
10-pvc.yaml	PersistentVolumeClaim	External block volume via the oci-bv CSI class
20-deployment.yaml	Deployment	NGINX serving content from the volume
30-service-lb.yaml	Service type=LoadBalancer	OCI Load Balancer provisioned by the cluster

5.1 The persistent volume claim

```
spec:
  storageClassName: oci-bv
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 50Gi
```

k8s/10-pvc.yaml

Three behaviours here look like faults and are not, and all three were anticipated in the manifest comments before the first apply:

- The claim sits Pending until a pod is scheduled. That is WaitForFirstConsumer working correctly — the volume is not cut until Kubernetes knows which availability domain to cut it in. Pending after the pod is Running would be a real failure; Pending before is expected.
- 50Gi is the OCI minimum. A smaller request is rounded up rather than rejected.
- lost+found appears in the web root. The CSI driver formats the raw device as ext4 on first attach, and ext4 creates it. Its presence is positive evidence that the mount is a real block device rather than a container filesystem layer.

5.2 The deployment

One replica, Recreate strategy — and that is a consequence of the storage, not a limitation of the application. A ReadWriteOnce block volume attaches to one node at a time, so a second replica scheduled elsewhere would sit Pending indefinitely. Recreate makes a rollout terminate the old pod, releasing the volume, before starting the new one, instead of deadlocking on the attachment.

An init container seeds the volume before NGINX starts. This is necessary because mounting a volume over /usr/share/nginx/html hides whatever the image shipped there, so without seeding the result is a 403 on an empty directory. The init container writes index.html and a seeded.txt carrying a timestamp — and that timestamp is what makes the persistence test in section 7.6 unambiguous.

It seeds only when seeded.txt is absent, and that guard is load-bearing rather than tidiness. Init containers run on every pod start, not only the first. An unconditional write would have rewritten the timestamp on the replacement pod, so the persistence test would have reported two different timestamps and appeared to fail — while the volume had in fact persisted perfectly. The evidence would have been destroyed by the act of gathering it.

5.3 The service and load balancer

```
annotations:
  oci.oraclecloud.com/load-balancer-type: "lb"
  service.beta.kubernetes.io/oci-load-balancer-shape: "flexible"
  service.beta.kubernetes.io/oci-load-balancer-shape-flex-min: "10"
  service.beta.kubernetes.io/oci-load-balancer-shape-flex-max: "10"
  service.beta.kubernetes.io/oci-load-balancer-internal: "false"
  service.beta.kubernetes.io/oci-load-balancer-security-list-management-mode: "None"
```

k8s/30-service-lb.yaml

The load balancer lands in whichever subnet the cluster was registered with as `service_lb_subnet_ids` — which Terraform set to the public lb subnet — so no explicit subnet annotation is needed. The final annotation is the one discussed in section 3.5, and is the least obvious line in the whole manifest set.

6. Implementation

6.1 Method

The work ran in phases, ordered so that the two long unattended waits — cluster creation and node pool creation — overlapped with the Kubernetes course reading rather than with idle time.

Phase	Activity	Note
0	Pre-flight: IAM, service limits, tooling	Blocking. Run before writing anything.
1	Terraform module theory and module code	Dynamic blocks, conditionals, reusability
2	Kubernetes course modules 1–5	Modules 6 and 7 deferred per the brief
3	terraform init / fmt / validate / plan / apply	Network layer first
4	Cluster and node pool creation	~25–35 min unattended
5	kubectl apply -f k8s/	PVC, deployment, load balancer
6	Verification and evidence capture	Four requirements plus the persistence test
7	Kubernetes course modules 8–9	
8	Documentation, diagram, this report	
9	Review and submission	
10	Teardown	kubectl delete first, then terraform destroy

6.2 Pre-flight

Week 2 was delayed twice by IAM permissions that only surfaced after the code was written — the account had read but not manage on bastion-family, and no access at all to the Run Command service — and recovering cost a rebuild of the instance. Week 3 therefore begins by proving the permissions exist before any code is applied.

The permissions this lab needs beyond Week 2 are cluster-family, log-groups, log-content, load-balancers and volume-family, plus the tenancy-level statement allowing the OKE service itself to manage resources. The pre-flight stage checks each one with a list call, on the theory that a missing policy returns `NotAuthorizedOrNotFound`.

That theory is only half right, and the run proved it. Every pre-flight check passed — including cluster-family — and cluster creation then failed anyway with `404-NotAuthorizedOrNotFound`. The reason is that a successful list proves only inspect or read on that resource type. It says nothing about manage, which is what creating requires. The account could see that zero clusters existed and could not make one, and no read-only probe can tell those two states apart.

OCI returns 404 rather than 403 deliberately: answering "forbidden" would confirm to an unauthorised caller that the resource exists. The cost of that design is that a permissions problem and a genuinely missing resource are indistinguishable from the error alone, which is exactly what makes this failure mode expensive to debug — the instinct is to go looking at the configuration.

So the pre-flight stage is worth keeping, but its guarantee is narrower than it was designed to be: it catches a compartment with no access at all, and it cannot catch read-without-manage. The only proof of manage is attempting a create. That limitation is now documented in the runbook rather than left as an assumption.

The pre-flight stage ran on 19 August 2026 and reported no failures. Tooling was resolved first; two of the three tools were not present on the machine at the start of the week.

```
terraform    Terraform v1.15.8
oci          3.90.3
kubect1      Client Version: v1.36.3
run-lab.sh preflight — tool versions
```

Each of the five services the lab touches was then exercised with a harmless list call in the target compartment. A NotAuthorizedOrNotFound response would have meant a missing policy statement; all five answered.

```
cluster-family      ok
log-groups          ok
load-balancers      ok
volume-family       ok
virtual-network-family ok
```

```
VM.Standard.E4.Flex : available
```

```
run-lab.sh preflight — IAM reachability in intern-04-andrew-hany-cmp
```

No policy had to be requested. Every statement anticipated in section 6.2 — cluster-family, log-groups, log-content, load-balancers and volume-family — was already granted, and the apply subsequently created all twenty-four resources without a single authorisation error. The pre-flight check cost fifteen minutes and, on this occasion, bought nothing. That is the correct outcome to report: the value of the check is insurance, and insurance that does not pay out has still done its job.

Service limits could not be read programmatically. The Limits API requires inspect limits at the tenancy level, which the intern group does not hold:

```
$ oci limits value list --service-name container-engine --compartment-id <tenancy>
ServiceError: NotAuthorizedOrNotFound (404)
```

Limits were therefore confirmed visually in the Console, and in any case the run itself is the stronger evidence: an OKE cluster, two E4.Flex instances with four OCPUs between them, one flexible load balancer and a 50 GB block volume were all created without hitting a quota.

6.3 Automation and evidence capture

The lab is driven by run-lab.sh, which executes each phase, saves the full output of every command to a timestamped evidence folder, and pauses at the exact moments a screenshot is worth taking, naming the file to use. It is plain POSIX shell, so the same script runs unchanged in OCI Cloud Shell.

Two checks in it are worth describing because they catch specific mistakes this design is prone to. After planning, the script parses terraform show -json and asserts the resource counts:

```
Assert-Count 'oci_core_subnet'          4
Assert-Count 'oci_core_route_table'      4
Assert-Count 'oci_core_security_list'    4
Assert-Count 'oci_logging_log'           4
Assert-Count 'oci_logging_log_group'     1    # <- not 5
Assert-Count 'oci_containerengine_cluster' 1
Assert-Count 'oci_containerengine_node_pool' 1
```

```
run-lab.sh
```

The log group count is the interesting one. It must be 1, because the root passes a single shared group into all four module calls and the modules therefore skip creating their own. A count of 5 would mean the `flow_logs_log_group_id` wiring had silently broken — a fault that produces perfectly working infrastructure and quietly defeats the design intent, which is exactly the kind that survives a manual review.

The script also reads the pod IP back after deployment and checks it falls inside the pod subnet, since an address in 10.244.0.0/16 would mean flannel rather than VCN-native networking — the single most important thing to get right in the whole lab.

The plan produced twenty-four resources and every count assertion passed on the second attempt — the first is described in section 6.4.

Plan: 24 to add, 0 to change, 0 to destroy.

```
OK    oci_core_subnet                4
OK    oci_core_route_table           4
OK    oci_core_security_list         4
OK    oci_logging_log                 4
OK    oci_logging_log_group           1
OK    oci_containerengine_cluster     1
OK    oci_containerengine_node_pool   1
OK    cni_type = OCI_VCN_IP_NATIVE
```

run-lab.sh plan — resource counts and the two assertions that matter

The log group assertion is worth more than the count it prints. Counting alone would be satisfied by one group created in the wrong place, so the plan was also checked at the address level. It contains `oci_logging_log_group.flow_logs[0]` in the root and four `module.subnet_*.oci_logging_log.flow[0]` entries, and no `module.subnet_*.oci_logging_log_group.this` at all — the modules are genuinely declining to create their own, which is the behaviour the shared-group design depends on.

The apply took nine minutes forty-four seconds in total, well inside the twenty-five to thirty-five minutes budgeted:

```
network layer (VCN, gateways, 4 subnets,
  4 route tables, 4 security lists,
  4 flow logs, 1 log group)           ~25s
oci_containerengine_cluster           6m 30s
oci_containerengine_node_pool         3m 01s
-----
apply duration                        9m 44s
```

evidence/2026-08-19_lab/tf06-apply.txt and tf07-apply-duration.txt

Both figures came in below the eight-to-fifteen and ten-to-twenty minute estimates in the runbook. The load balancer was faster still: the cloud-controller-manager logged `EnsuringLoadBalancer` at 00:32:54Z and `EnsuredLoadBalancer` at 00:33:10Z — sixteen seconds, against a documented expectation of three to five minutes.

A second plan was run after the apply to check for drift. It reported no changes, which is the property the design was aiming for and is discussed further in section 6.4.

No changes. Your infrastructure matches the configuration.

6.4 Problems encountered

Ten problems were recorded across the week. Four of them are worth writing up in full, because each says something about the system rather than about the laptop: two were defects in this repository that reading alone had not found, and two only existed once the code met a real provider or a real worker node. The other six were environmental, and are summarised at the end. The complete record, with exact error text, is in the lab log.

terraform plan failed with Invalid count argument

Symptom: terraform validate passed, and the first real plan failed four times over — once per subnet module call.

Error: Invalid count argument

```
on modules\subnet\main.tf line 230, in resource "oci_logging_log_group" "this":
230:   count = var.flow_logs_enabled && var.flow_logs_log_group_id == null ? 1 : 0
```

The "count" value depends on resource attributes that cannot be determined until apply, so Terraform cannot predict how many instances will be created.

the first plan against a real Terraform binary

Diagnosis: count must be resolvable at plan time. The module inferred whether the caller had supplied a log group by testing the argument against null, but the root passes oci_logging_log_group.flow_logs[0].id — the OCID of a log group being created in the same run, whose value at plan time is (known after apply). Comparing an unknown value to null yields an unknown boolean, and count cannot accept one. The design would have planned cleanly if a literal OCID had been typed into terraform.tfvars, which is exactly why static review missed it.

Fix: drive the conditional from a plan-time-known boolean instead. The module already did this twice, so the log group now follows the convention its two siblings were already using:

route table	create_route_table	route_table_id
security list	create_security_list	security_list_ids
flow log group	create_flow_logs_log_group	flow_logs_log_group_id

What it revealed: this is the sharpest lesson of the week, and it is about the plan/apply boundary rather than about OCI. Inferring intent from an argument's value couples the conditional to when that value becomes available; a separate flag does not. It also showed the limit of terraform validate, which checks syntax, types and provider schema without evaluating anything that depends on real resources — unknown-ness does not exist until there is a graph to walk. A configuration that validates is not a configuration that plans.

The pod would not start: cri-o short-name enforcement

Symptom: the claim bound, the volume attached, the init container completed and the load balancer came up, but the nginx container never started.

```
Failed to inspect image "": rpc error: code = Unknown desc = short name mode
is enforcing, but image name nginx:1.27-alpine returns ambiguous list
```

Diagnosis: OKE worker nodes run cri-o, not Docker, and Oracle Linux ships registries.conf with short-name-mode set to enforcing. Docker silently assumes Docker Hub for an unqualified name; cri-o refuses to guess when a short name could resolve against more than one configured registry. The misleading detail is that the init container's busybox:1.36 had pulled successfully three seconds earlier from the same node, because busybox has an entry in the shipped short-name alias table and nginx does not.

Fix: fully qualify both images as docker.io/library/nginx:1.27-alpine and docker.io/library/busybox:1.36. What it revealed: a failure that only a real node could produce. kubectl accepted the manifest, the scheduler placed the pod and the volume attached — everything Kubernetes-side succeeded, and the runtime failed afterwards. It also showed that one container succeeding is not evidence that a sibling's image reference is well formed.

A permanently dirty plan after a successful apply

Symptom: with nothing changed, the post-apply plan reported 0 to add, 18 to change, 0 to destroy — every resource both modules create, wanting an update, indefinitely.

Diagnosis: not the load balancer. The oci-load-balancer-security-list-management-mode annotation described in section 5 did exactly its job and no security list rule drifted at all. All eighteen diffs were the same attribute:

```
module.oke.oci_containerengine_cluster.this
```

```
before: {"Oracle-Tags.CreatedBy": "ejada-interim-program/...",
        "Oracle-Tags.CreatedOn": "2026-08-20T00:21:59.239Z"}
after : {}
```

The tenancy has the Oracle-Tags default namespace enabled, so OCI stamps CreatedBy and CreatedOn onto every resource at creation. Both modules passed `defined_tags = var.defined_tags`, whose default is an empty map — and an empty map is not an absence of opinion, it is an instruction to remove every defined tag. Terraform proposed stripping tags that OCI immediately reapplies. The giveaway was in the same state file: `oci_core_vcn` in the root never sets `defined_tags`, carries the same two Oracle tags, and showed no drift.

Fix: send null rather than an empty map when the caller supplied nothing. The provider schema settles what that means — `defined_tags` is Optional and Computed, the combination in which null means leave this to the provider.

```
defined_tags = length(var.defined_tags) > 0 ? var.defined_tags : null
```

The module contract is unchanged: same variable, same default, and a caller who does pass defined tags still gets them applied. The plan afterwards reports no changes. What it revealed: in Terraform an empty collection and an absent value are different instructions. More generally, a clean plan after apply is a property that has to be designed for, because anything else that writes to the resources Terraform owns — the cloud-controller-manager, or a tenancy's default tag namespace — will otherwise contest them on every run.

The persistence test could not have proved anything

Symptom: none. This one was found by reading the manifest, before the cluster existed.

Diagnosis: the init container in the Deployment wrote `seeded.txt` with the current time on every start, and an init container runs on every pod start. The replacement pod in the persistence test would therefore have stamped a new timestamp onto the volume, and the test would have reported two different times.

Fix: seed only when the file is absent, so the volume is written once and left alone afterwards. The runner also compares the two readings itself and requires the timestamps to match and the pod name to differ, rather than leaving the comparison to whoever is reading the screen.

What it revealed: this was the most dangerous defect of the four, and the only one that would have failed silently. It would have run to completion, produced clean-looking output, and demonstrated the opposite of its stated conclusion — that the data did not survive the pod. A test that can only pass is not evidence; this one had to be made capable of failing before its passing meant anything. It is also the only defect here that no amount of running would have surfaced, because nothing about it errors.

The six environmental problems

None of the remaining six involved OCI, Terraform or Kubernetes, and together they took longer than building the cluster. The OCI CLI would not install, because the longest path inside its package is 229 characters and Oracle's installer targets a directory deep enough to push that past the 260-character Windows limit; installing into a short path solved it, with no administrator rights and no reboot. `kubect` was absent. There was no OCI CLI configuration file at all, Weeks 1 and 2 having been done in the pre-authenticated Cloud Shell, so the key fingerprint was derived from the private key already on disk rather than looked up. Git Bash rewrote absolute paths inside `kubect` exec, so container paths arrived resolved against the Git installation and had to be exempted from that conversion. An antivirus false positive blocked the PowerShell runner, which is why the run recorded here used the bash one. And terraform `fmt` wanted four files realigned. The lesson from the set is in section 10.1: prove the toolchain end to end before writing any infrastructure code, because none of this was visible from the assignment.

7. Verification

The brief asks for four things. Each is demonstrated below with evidence rather than asserted.

7.1 Summary

Requirement	Evidence	Result	Figure
OKE cluster with VCN-native pod networking	Pod IP inside 10.0.32.0/19		
Managed worker node pool	Console node pool details; kubectl get nodes		
External block volume attached	df -h at the mount point; Console shows Attached		
Exposed through a load balancer	curl of the external IP; healthy backends		
Data survives pod deletion	Identical seeded.txt across two pods		
Flow logs on all four subnets	Console Logging; Log Search returns records		

7.2 VCN-native pod networking

The decisive evidence is a single field: the pod IP. Under VCN-native networking it is an address from the pod subnet, allocated out of the VCN itself; under flannel it would be an overlay address from 10.244.0.0/16 with no existence in OCI. The address can be corroborated on the OCI side by listing private IPs in the pod subnet and finding the same one.

The pod holds a real VCN address drawn from the dedicated pod subnet.

```
NAME                                READY   STATUS    IP             NODE
nginx-demo-78b45748dc-p7sfn        1/1     Running   10.0.33.44     10.0.1.147
kubectl get pods -n ejada-w3 -o wide — evidence/2026-08-19_lab/ver01-pods-wide.txt
```

10.0.33.44 falls inside the pod subnet 10.0.32.0/19. This single field is the requirement: under FLANNEL_OVERLAY the pod would carry an address from 10.244.0.0/16, an overlay range that has no existence in the VCN and cannot be routed to, filtered by a security list, or seen in a flow log. The address here is a VCN IP with all three properties.

OKE corroborates it independently through the node labels it applies itself, which is stronger evidence than the address alone because it reflects the cluster's own view of its CNI:

```
oci.oraclecloud.com/vcn-native-ip-cni=true
oke.oraclecloud.com/node.info.private_worker=true
kubectl get nodes --show-labels — evidence/2026-08-19_lab/ver04-node-labels.txt. The Console reports the same as Network
type: OCI_VCN_IP_NATIVE — screenshots/console/fig12-cluster-details.png
```

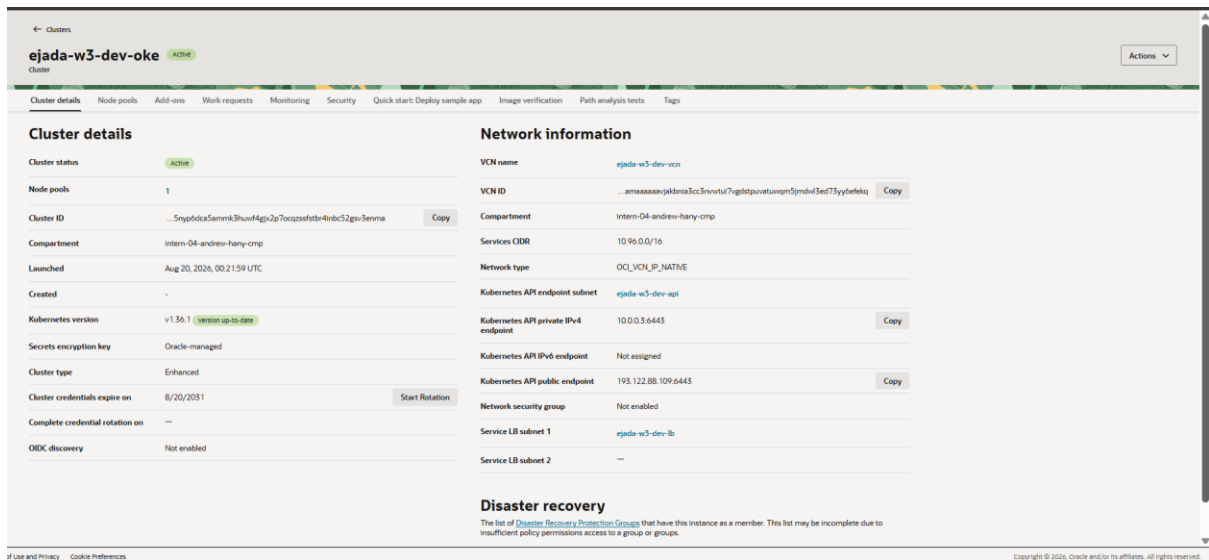


Figure 2. Cluster *ejada-w3-dev-oke*. Network type *OCI_VCN_IP_NATIVE*, Kubernetes *v1.36.1*, Enhanced, services CIDR *10.96.0.0/16*, and a public API endpoint at *193.122.88.109:6443*. The API endpoint, pod and load balancer subnets are each named.

7.3 Managed worker node pool

The node pool is managed, not virtual: OKE provisions the instances, and OCI reports them as cluster members.

NAME	STATUS	ROLES	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE
10.0.1.147	Ready	node	v1.36.1	10.0.1.147	<none>	Oracle Linux 8.10
10.0.1.246	Ready	node	v1.36.1	10.0.1.246	<none>	Oracle Linux 8.10

kubectl get nodes -o wide — evidence/2026-08-19_lab/ver03-nodes.txt

Both addresses sit inside the worker subnet *10.0.1.0/24* and neither node has an external IP, which is the intended posture: the load balancer is the only path in from the internet. The label *oci.oraclecloud.com/node.info.managed=true* confirms the pool type from OKE's own side, and *workload=general* is the label the module's initial_node_labels dynamic block emitted, so the block is demonstrably doing work rather than merely being present in the code.

Neither the Kubernetes version nor the node image is written down anywhere in the configuration. Both were discovered at plan time — the module reads *oci_containerengine_cluster_option* for the newest version the region offers, then filters *oci_containerengine_node_pool_option.sources* for an image matching that version, the pool's CPU architecture and no GPU variant.

```
kubernetes_version    v1.36.1
node image            ocid1.image.oc1.me-jeddah-1.aaaaaaaawv57iy2lnhx6j22eb
                     7uj6qyjoqabfumwu75eew7v6vt5h5d3pjoa
shape                 VM.Standard.E4.Flex, 2 OCPU / 16 GB, size 2
placement             ME-JEDDAH-1-AD-1, fault domains 1 and 2
terraform output kubernetes_version / node_pool_images
```

The version has since been pinned to *v1.36.1* in *terraform.tfvars*. Leaving it resolving to newest is right for a first build and wrong to keep, for the reason given in section 10.1.

The Console confirms the same facts from OCI's side, and adds one the OCID alone does not show — the image's name:

Node type	Managed
Shape	VM.Standard.E4.Flex
OCPUs / memory	2 / 16 GB
Custom boot volume size	60 GB
Total worker nodes	2
Image name	Oracle-Linux-8.10-2026.06.15-0-OKE-1.36.1-1505

```

Placement AD 1      ME-JEDDAH-1-AD-1
Worker node subnet  ejada-w3-dev-workers
Pod communication subnet ejada-w3-dev-pods
Number of pods per node 31

```

Console — node pool ejada-w3-dev-oke-app

The image name closes the loop on the discovery logic. The module filters `oci_containerengine_node_pool_option.sources` with a regex on `"OKE-${local.version_tag}"`, where `version_tag` is the cluster version with the leading `v` stripped. The image OKE chose is literally named `Oracle-Linux-8.10-2026.06.15-0-OKE-1.36.1-1505`, so the OKE-1.36.1 the filter matched on is visible in the result. Nothing about that string appears in the configuration.

Two other fields are the module's inputs arriving intact. Pod communication subnet is the second subnet a VCN-native node pool needs and the reason `node_pool_pod_network_option_details` exists as a conditional block; a flannel cluster would show nothing there. Number of pods per node is the module's `max_pods_per_node` default of 31, which caps how many VCN addresses each node may consume from the /19.

The screenshot shows the Oracle Cloud console interface for the node pool **ejada-w3-dev-oke-app**. The left sidebar contains navigation tabs: Details, Kubernetes labels, VNICs, Security, Monitoring, Work requests, and Tags. The main content area is divided into four sections:

- Node pool status:** Shows the node pool is **Active**. Fields include Node pool ID, Node type (Managed), Compartment (intern-04-andrew-hany-cmp), Network security groups (None), Custom boot volume size (60), OCPUs (2), Amount of memory (GB) (16), Kubernetes version (v1.36.1, Version up-to-date), Shape (VM.Standard.E4.Flex), Image name (Oracle-Linux-8.10-2026.06.15-0-OKE-1.36.1-1505), and Total worker nodes (2).
- Placement configurations:** Shows AD 1 (oXVtME-JEDDAH-1-AD-1), Worker node subnet (ejada-w3-dev-workers), and Capacity type (On-demand capacity).
- Pod communication:** Shows Subnet (ejada-w3-dev-pods), Network security groups (None), and Number of pods per node (31).
- Node cycling information:** Shows Cycling option (Replace nodes) and Maximum surge (—).

Figure 3. Node pool *ejada-w3-dev-oke-app*. Node type *Managed*, *VM.Standard.E4.Flex* at 2 OCPU / 16 GB, and — the field that matters — a Pod communication subnet distinct from the worker node subnet, at 31 pods per node.

The screenshot shows the Oracle Cloud console interface for the cluster **ejada-w3-dev-oke**. The left sidebar contains navigation tabs: Cluster details, Node pools, Add-ons, Work requests, Monitoring, Security, Quick start: Deploy sample app, Image verification, Path analysis tests, and Tags. The main content area is titled **Node pools** and includes a search bar, an 'Add node pool' button, and a table of node pools.

Node pool	Node type	Node pool status	Kubernetes version	Kubernetes status	Image name	Total worker nodes
ejada-w3-dev-oke-app	managed	Active	v1.36.1	Version up-to-date	Oracle-Linux-8.10-2026.06.15-0-OKE-1.36.1-1505	2

Page 1 of 1 (1 - 1 of 1 total items) | Items per page: 10

Figure 4. The node pool listed against the cluster: managed, Active, v1.36.1, two worker nodes, running image Oracle-Linux-8.10-2026.06.15-0-OKE-1.36.1-1505.

7.4 External block volume

Three independent pieces of evidence are collected: df at the mount point showing a ~50 G device rather than the container overlay filesystem; a directory listing showing the seeded files alongside lost+found; and the volume itself in the Console in state Attached, bound to a specific worker node.

The claim bound and the volume was created, attached and formatted by the CSI driver.

```
NAME                                CAPACITY  ACCESS  STORAGECLASS  STATUS
csi-6458d903-604b-4786-8317-1fcb985c4416  50Gi      RWO     oci-bv         Bound
kubectl get pvc,pv -n ejada-w3 — evidence/2026-08-19_lab/app03-pvc.txt
```

Inside the pod, the mount is a real block device rather than a container filesystem layer:

```
Filesystem      Size  Used Available Use% Mounted on
/dev/sdb        48.9G  32.0K    48.9G    0% /usr/share/nginx/html

total 28
drwxr-xr-x  3 root root    4096 Aug 20 00:33 .
-rw-r--r--  1 root root     777 Aug 20 00:33 index.html
drwx-----  2 root root   16384 Aug 20 00:33 lost+found
-rw-r--r--  1 root root      39 Aug 20 00:33 seeded.txt
kubectl exec — df -h and ls -la — evidence/2026-08-19_lab/ver05-df.txt, ver06-ls.txt
```

Two details in that listing carry the argument. The device is /dev/sdb with 48.9 G available, not the overlay filesystem the container image would otherwise present. And lost+found is present — a directory that only exists because mkfs.ext4 created it, so its presence is direct evidence that the CSI driver formatted a freshly attached block device rather than this being a directory baked into the image.

OCI's own view agrees. The volume exists in the compartment as a first-class resource and is attached over iSCSI to the worker node the pod was scheduled on:

```
volume      ocid1.volume.oc1.me-jeddah-1.abvgkljr4hhywttfzi3wanwak
             krzvwi6xk5kcnuhpzq4wjed6vsd43pj4wq
size        50 GB
attachment  ATTACHED, iscsi, /dev/oracleoci/oraclevdb
oci bv volume list / oci compute volume-attachment list
```

The storage class oci-bv uses volumeBindingMode WaitForFirstConsumer, so the claim deliberately stayed Pending until a pod was scheduled. That is not a fault: it is what lets OCI cut the volume in the same availability domain as the node that will mount it. A claim still Pending after its pod is Running would be a genuine failure; a claim Pending before that is the design working.

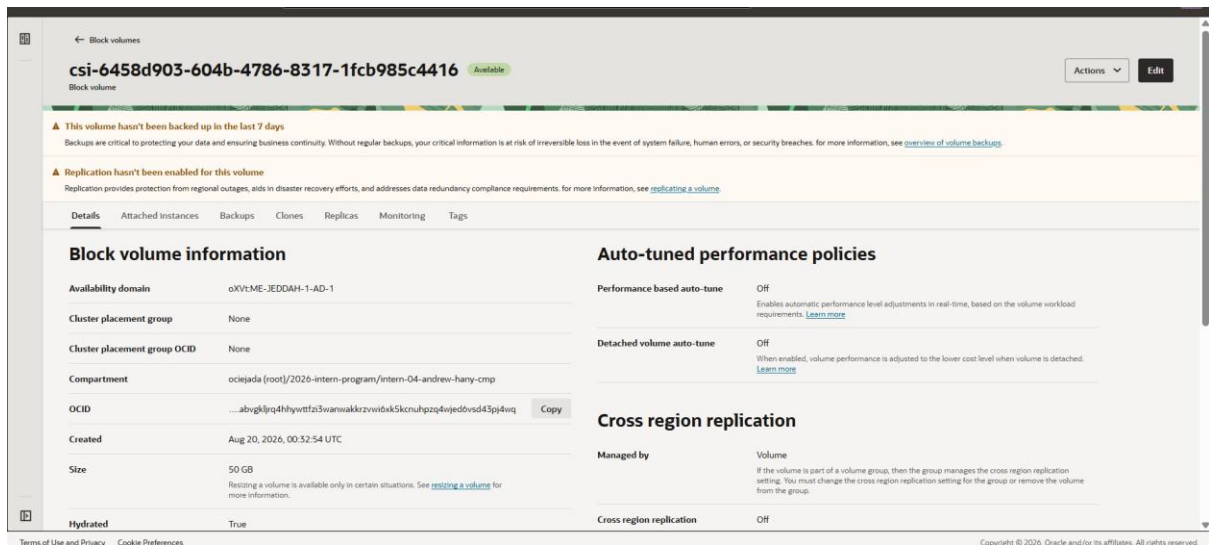


Figure 5. The 50 GB block volume the CSI driver created from the PersistentVolumeClaim, in availability domain ME-JEDDAH-1-AD-1.

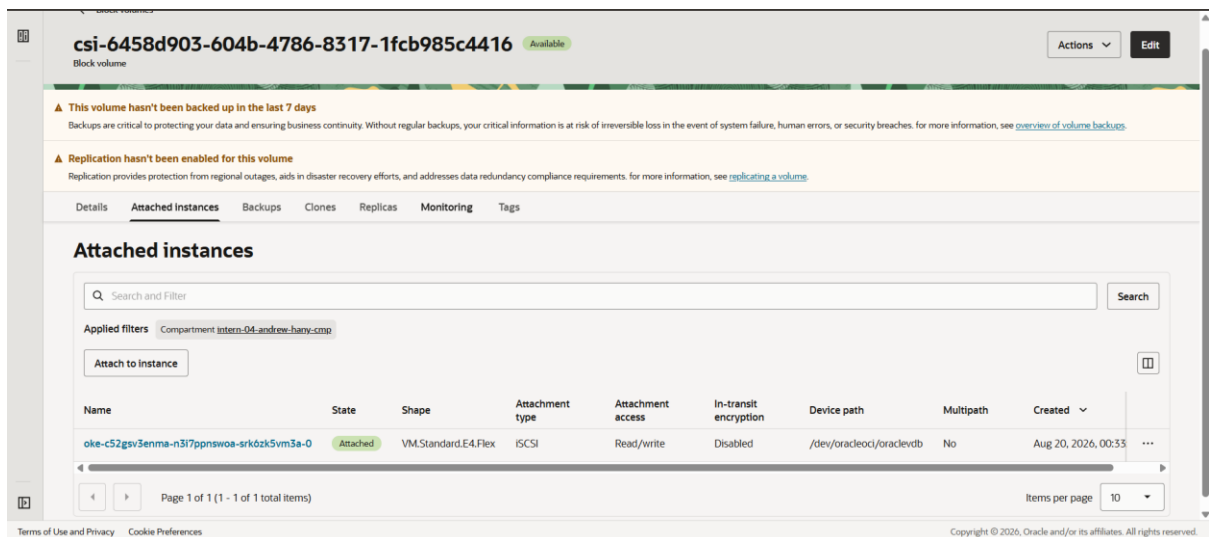


Figure 6. The same volume attached over iSCSI to worker node oke-c52gsv3enma-n3i7ppnswoa-srk6zk5vm3a-0 at /dev/oracleoci/oraclecvdb, read/write.

7.5 Load balancer exposure

The Service of type LoadBalancer caused the OCI cloud-controller-manager to provision a real load balancer in the public LB subnet.

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
nginx-demo	LoadBalancer	10.96.120.122	144.24.222.84	80:32222/TCP

NodePort 32222/TCP
Endpoints 10.0.33.44:80

kubectl get svc / describe svc — evidence/2026-08-19_lab/app06-svc.txt, ver08-svc-describe.txt

The full path is exercised end to end from outside the VCN:

```
$ curl -s -i http://144.24.222.84
```

```
HTTP/1.1 200 OK
Server: nginx/1.27.5
Content-Type: text/html
Content-Length: 777
Last-Modified: Thu, 20 Aug 2026 00:33:42 GMT
```

evidence/2026-08-19_lab/ver09-curl.txt

The Last-Modified header is worth noticing: it is the moment the init container wrote the file to the block volume, so a single HTTP response demonstrates the load balancer, the NodePort, the pod and the mounted volume at once.

OCI reports the backend set healthy on both nodes:

```
shape                flexible, 10 / 10 Mbps, public
ip-address           144.24.222.84
status               OK
total-backend-count  2
critical-state-backends []
```

oci lb backend-set-health get

The Service carries the annotation `oci-load-balancer-security-list-management-mode` set to `None`, for the reason given in section 5.3. Its effect was confirmed rather than assumed: the post-apply plan showed no drift on any route table or security list, so the cloud-controller-manager did indeed write nothing into the rules Terraform owns.

The screenshot shows the OCI Load Balancer console for a specific load balancer. The 'Load balancer health' tab is active, displaying an 'Overall health' of 'OK'. Below this, a table shows the 'Backend sets health' with counts for Critical (0), Warning (0), Incomplete (0), Pending (0), and OK (1). The 'Backend sets drain status' shows 'Drained' with a count of 0. The 'Load balancer information' tab is also visible, showing details like OCID, Created date (Aug 20, 2026), Shape (Flexible), Min/Max bandwidth (10 Mbps), IP address (144.24.222.84), Virtual cloud network (ejada-w3-dev-vcn), and Subnet (ejada-w3-dev-lb).

Figure 7. The load balancer created by the Service: flexible shape at 10/10 Mbps, public IP 144.24.222.84 in the LB subnet, Overall health OK.

The screenshot shows the 'Backend sets' tab in the OCI Load Balancer console. It features a search bar and a 'Create backend set' button. Below is a table with columns: Name, Cipher suite, Traffic distribution policy, Number of backends, Drained, % of backends drained, and Health. The table contains one entry: 'TCP-80' with a cipher suite of '-', a weighted round robin policy, 2 backends, 0 drained, 0% drained, and a health status of 'OK'. The page indicates it is 'Page 1 of 1 (1 - 1 of 1 total items)' and has a 'Items per page' dropdown set to 25.

Figure 8. Backend set TCP-80 — two backends, none drained, health Ok. The two backends are the two worker nodes.

7.6 The persistence test

This is the test worth more than the screenshots. The seeded file is read, the pod is deleted, the replacement pod is read, and the two are compared.

```
kubectl exec -n ejada-w3 deploy/nginx-demo -- cat /usr/share/nginx/html/seeded.txt
kubectl delete pod -n ejada-w3 -l app=nginx-demo
# wait for the replacement
kubectl exec -n ejada-w3 deploy/nginx-demo -- cat /usr/share/nginx/html/seeded.txt
```

A different pod name with an identical timestamp is the proof: the data was not rebuilt, it was still there. It lives on the block volume, not in the image, and the volume outlived the pod that was using it.

Two properties make this a real test rather than a demonstration. It is falsifiable — had the CSI driver handed the replacement pod a fresh volume, the timestamp would differ and the failure would be visible rather than silent. And it is checked mechanically: the runner compares the two captured files and requires the timestamps to match AND the pod name to have changed, so neither half can pass on its own. A matching timestamp with an unchanged pod name would mean the pod never restarted, which proves nothing at all.

The pod was deleted and the Deployment replaced it. The file on the volume was read before and after.

```
pod before deletion  nginx-demo-78b45748dc-p7sfn
pod after  deletion  nginx-demo-78b45748dc-xmnpj5

seeded.txt, before   seeded at Thu Aug 20 00:33:42 UTC 2026
seeded.txt, after    seeded at Thu Aug 20 00:33:42 UTC 2026
evidence/2026-08-19_lab/per01-seeded-before.txt, per03-seeded-after.txt, per05-summary.txt
```

A different pod, an identical timestamp. The file was not regenerated — it was still there. Had the content lived in the container image, the timestamp would have moved, because a new container would have written it again; had it lived in the pod's ephemeral storage it would have vanished with the pod. It survived because it lives on a block volume whose lifecycle is bound to the PersistentVolumeClaim rather than to any pod.

The result is in fact stronger than the test was designed to show. The timestamp 00:33:42 was written by an earlier pod still — nginx-demo-8698c8867f-gkxjr, the one that failed to start because of the image problem in section 6.4 — so by the time the formal test ran, the data had already survived a pod replacement and a change of ReplicaSet caused by editing the Deployment. Three pods, across two ReplicaSets, one timestamp.

That outcome depends entirely on the init container seeding the volume only when seeded.txt is absent. Without that guard every replacement pod would rewrite the file, the timestamps would differ, and the test would have appeared to disprove exactly what it was built to demonstrate.

7.7 Flow logs

Flow logging is the fourth resource in the Subnet module and the easiest to forget to verify, since nothing else in the lab depends on it working.

All four subnets emit flow logs, and all four write into the single log group created by the root configuration.

```
enabled  name                                retention  state
True     ejada-w3-dev-api-flowlog                30        ACTIVE
True     ejada-w3-dev-workers-flowlog            30        ACTIVE
True     ejada-w3-dev-pods-flowlog                30        ACTIVE
True     ejada-w3-dev-lb-flowlog                  30        ACTIVE
oci logging log list --log-group-id <ejada-w3-dev-flow-logs>
```

Four logs, one group. This is the visible outcome of the module's fourth resource and of the shared-log-group wiring described in section 4.4: each of the four module calls created its log, and none created a group, because the root supplied one. One Log Search therefore covers the whole VCN rather than four searches covering a quarter of it each.

The LB subnet's log is the one to open, since it is the subnet with real external traffic — records there correspond to the curl in section 7.5.

Opening the LB subnet's log and running Explore with Log Search returns real records, which is the difference between flow logs being configured and flow logs working:

```
ACCEPT TCP 10.0.2.188 Port 5556 -> 10.0.1.246 Port 10256 Bytes 336 Packets 5
ACCEPT TCP 10.0.2.188 Port 9532 -> 10.0.1.147 Port 10256 Bytes 336 Packets 5
ACCEPT TCP 10.0.1.246 Port 10256 -> 10.0.2.188 Port 5556 Bytes 712 Packets 5
REJECT TCP 147.185.132.26 Port 53719 -> 10.0.2.161 Port 46342 Bytes 44 Packets 1
REJECT TCP 35.203.210.229 Port 50878 -> 10.0.2.161 Port 9628 Bytes 44 Packets 1
```

ejada-w3-dev-lb-flowlog, Explore log — 20 Aug 2026 01:59 UTC

Both halves of that sample are worth reading. The ACCEPT lines are the load balancer, at 10.0.2.188 in the LB subnet, health-checking kube-proxy on both worker nodes at 10.0.1.246 and 10.0.1.147 on port 10256 — traffic permitted by exactly one rule, the load-balancer egress entry in locals.tf, and by its matching ingress rule on the worker subnet. The reply is logged too.

The REJECT lines are unsolicited traffic from the public internet arriving at the load balancer's subnet on ports 46342 and 9628 and being dropped, because the LB security list admits only TCP 80. Those are opportunistic scans, and they began within minutes of the load balancer acquiring a public address. They are the most useful lines in the log: a rule that permits the right traffic is only half a control, and this is the half that is normally invisible. The security list is demonstrably denying, not merely configured to deny.

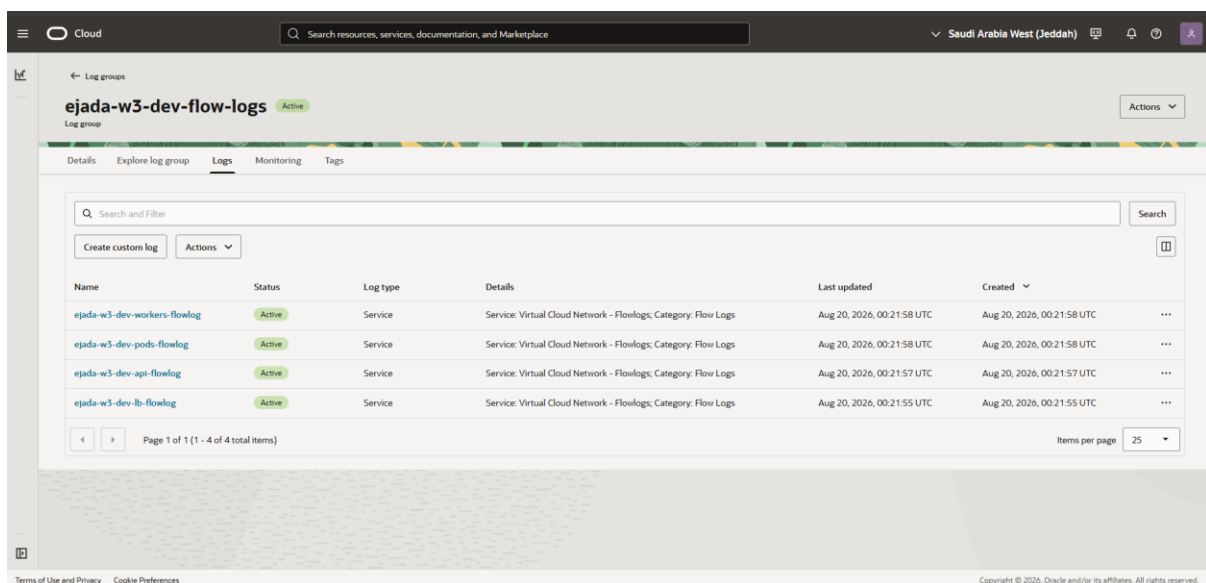


Figure 9. Log group *ejada-w3-dev-flow-logs* holding four flow logs, one per subnet, all Active. Four logs in one group is the shared-log-group wiring working.

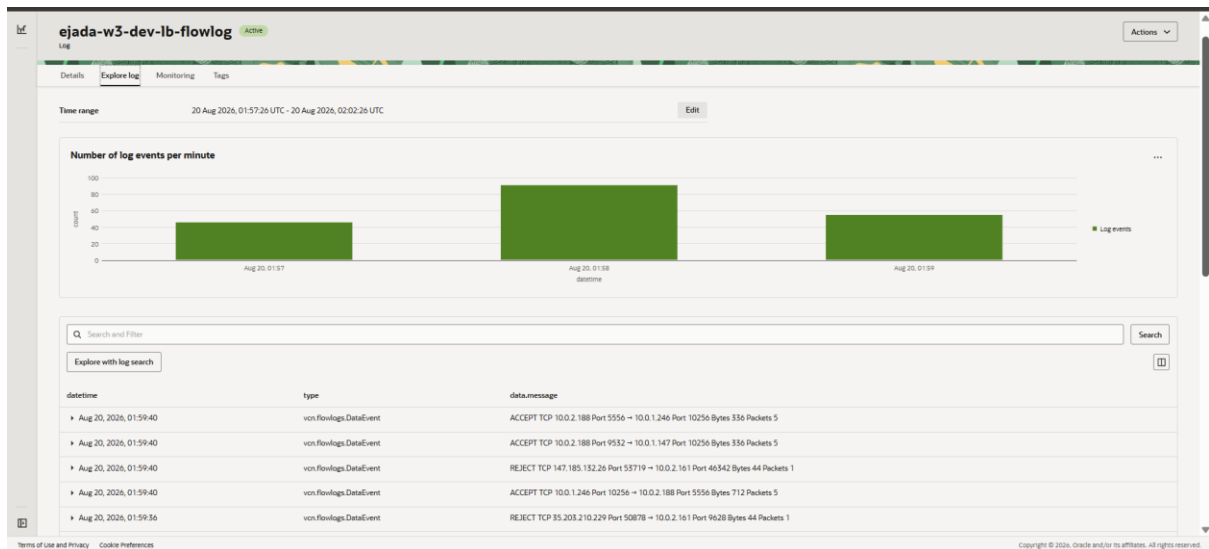


Figure 10. Flow records from the LB subnet. The ACCEPTs are load-balancer health checks reaching kube-proxy on both workers; the REJECTs are internet scans being dropped.

8. Security review

Control	Status in this build
Worker nodes have no public IPs	Enforced by <code>is_public = false</code> on the workers and pods subnets.
Load balancer is the only ingress path	Only the LB subnet may reach the NodePort range.
Private egress to OCI services	Service gateway; OKE and registry traffic never crosses the internet.
Secrets kept out of version control	<code>terraform.tfvars</code> , <code>*.pem</code> , <code>*.key</code> and <code>kubeconfig</code> are all gitignored.
Kubeconfig kept out of state	Not emitted as an output; the module outputs the generating command.
Default security list closed	Adopted by Terraform with no rules, rather than left with SSH open to the world.
VCN flow logs enabled	All four subnets, into one shared log group.
API endpoint source range	Currently <code>0.0.0.0/0</code> — see below.

The one control deliberately left open is the API endpoint source range. `kubectl_source_cidr` defaults to `0.0.0.0/0` so that the lab can be driven from a laptop or Cloud Shell without further setup. It is the first thing to narrow in any deployment that is not a lab, and it is a variable precisely so that narrowing it to a single `/32` is a one-line change rather than a code change.

Two further hardening steps were considered and left out as beyond the brief: a private API endpoint reached through a bastion, and envelope encryption of Kubernetes secrets with a customer-managed Vault key. The OKE module already accepts `kms_key_id` for the second.

9. Cost and teardown

The stack bills continuously while it exists. The teardown order is not optional, for the reason given in section 3.5: the load balancer and the block volume were created by Kubernetes, Terraform has no record of them, and the load balancer's VNIC prevents the lb subnet from being deleted.

```
kubectl delete -f k8s/           # releases the load balancer and the block volume
# confirm in the Console that both have disappeared
terraform destroy                # then, and only then
```

Four billable resources ran for the duration of the lab:

```
OKE cluster (enhanced)           created 2026-08-20 03:28 +03:00
node pool, 2 x VM.Standard.E4.Flex created 2026-08-20 03:31 +03:00
  (2 OCPU / 16 GB each, 60 GB boot)
load balancer, flexible 10 Mbps   created 2026-08-20 03:35 +03:00
block volume, 50 GB              created 2026-08-20 03:33 +03:00
```

creation times from the apply log and the OCI Console

The cluster was kept running after the evidence was captured so that it could be inspected live if asked. Teardown order is not a matter of preference. The load balancer and the block volume were created by Kubernetes rather than by Terraform, so they appear in no state file, and the load balancer holds a VNIC in the LB subnet which will block that subnet's deletion. The Kubernetes objects must therefore go first:

```
kubectl delete -f k8s/           # wait for the LB and the volume to disappear
cd terraform
terraform destroy
```

the order run-lab.sh teardown enforces, pausing between the two halves

After the destroy, the Console was checked for orphans in the four categories that survive a careless teardown: block volumes, node pool boot volumes, load balancers and log groups. Each of these continues to bill after the cluster it belonged to has gone.

10. Conclusion

The brief distinguished making it work from building it well, and the second is where the effort went. Two modules were built: one that turns a subnet, its routing, its filtering and its logging into a single reusable unit, and one that turns an OKE cluster and its node pools into another. Between them they use dynamic blocks, the zero-or-one idiom for conditional single blocks, count and for_each as conditionals, optional() object attributes, and validation and precondition guards — each because it solved a problem in this configuration, not as a demonstration.

The measurable outcome is that four subnets differing in size, posture, routing and rule count are produced by four calls to one module, with no branch inside that module written for any of them; and that changing the address plan in terraform.tfvars re-wires every security rule in the cluster without editing a single resource block.

The lab met all four requirements and the infrastructure came up more smoothly than expected — nine minutes forty-four seconds of apply against a budget of twenty-five to thirty-five, no IAM gaps, and worker nodes Ready ninety seconds after the node pool finished, which means the security list rules were right first time. What did not go smoothly was everything either side of that.

The honest summary is that the infrastructure code was in better shape than the environment around it, and that the two defects which did exist in the code were both of a kind that reading cannot find. The count bug could not have been caught by terraform validate, because validate never evaluates anything that depends on a real resource. The defined_tags drift could not have been caught by a first apply, because it only appears on the second plan. Both needed a real provider and a real tenancy, and neither was a mistake of carelessness — they were assumptions that were reasonable until tested.

The one that would have been genuinely damaging was found by reading rather than by running: the persistence test as originally written would have re-seeded the volume on every pod start and reported two different timestamps. It would have run to completion, produced clean-looking output, and demonstrated the opposite of its stated conclusion. That is a more dangerous class of defect than anything that fails loudly, and it is the reason the runner now compares the two timestamps itself instead of leaving the comparison to whoever is reading the screen.

10.1 What I would do differently

Pin the Kubernetes version from the beginning. Leaving kubernetes_version null and resolving to the newest version the region advertises is convenient for a first build and a liability afterwards: the value is updatable on both the cluster and the node pool, so the day OKE publishes a newer release an unrelated apply would propose a control-plane upgrade nobody asked for. It has now been pinned to v1.36.1, but that should have been the second commit rather than an afterthought.

Run terraform plan against a real tenancy far earlier. Everything in this repository was written and reviewed statically, and the review was good enough to catch nothing that mattered — the two real defects both needed a plan and an apply to surface. A plan costs nothing and creates nothing. Waiting until the code was finished before running one was the single largest process mistake of the week.

Distrust an empty collection. Both defects share a root: assuming that an absent value and an empty one mean the same thing. defined_tags = {} means remove all tags, not leave tags alone; and testing an argument against null silently assumes that argument's value is known at plan time. I would now treat any conditional written against the value of an input as a question about when that value becomes available, not merely about what it contains.

Check the container runtime's conventions rather than Docker's. Assuming `nginx:1.27-alpine` would resolve cost a debugging cycle on a cluster that was otherwise fully working. Fully-qualified image references are correct on every runtime and ambiguous on none, and there is no reason to write the short form.

Budget for the environment, not just the work. Four of the ten problems had nothing to do with OCI, Terraform or Kubernetes: a path-length limit, an antivirus false positive, a missing credential file and a shell rewriting arguments. Together they took longer than building the cluster. Next time I would get the toolchain proven end to end — install, authenticate, run one harmless command against the real tenancy — before writing any infrastructure code at all.

Appendix A — Repository contents

```

week_3_submission/
  README.md          overview and quick start
  run-lab.sh         lab runner + evidence capture
docs/
  TERRAFORM_MODULES_NOTES.md  dynamic blocks, conditionals, reusability
  ARCHITECTURE.md             address plan, traffic paths, decisions
  RUNBOOK.md                 every command, in order
  architecture.svg / .png / .drawio
terraform/
  modules/subnet/            subnet + route table + security list + flow log
  modules/oke/              cluster + node pools + add-ons
  network.tf                VCN, gateways, shared log group
  locals.tf                 all security and routing rules
  subnets.tf               4 x the subnet module
  oke.tf                    1 x the OKE module
  variables.tf outputs.tf provider.tf terraform.tfvars.example
k8s/
  00-namespace.yaml 10-pvc.yaml 20-deployment.yaml 30-service-lb.yaml
screenshots/console/ 21 Console captures (Figures 2-22)
evidence/2026-08-19_lab/ 42 captured command outputs from the run

```

Appendix B — Command reference

Command	Purpose
<code>.\run-lab.ps1 -Stage preflight</code>	IAM, limits and tooling checks
<code>.\run-lab.ps1 -Stage plan</code>	init, fmt, validate, plan, resource-count assertions
<code>.\run-lab.ps1 -Stage apply</code>	apply and capture the outputs
<code>.\run-lab.ps1 -Stage kubeconfig</code>	generate kubeconfig, wait for nodes Ready
<code>.\run-lab.ps1 -Stage deploy</code>	apply the manifests, wait for the load balancer
<code>.\run-lab.ps1 -Stage verify</code>	the four requirement checks
<code>.\run-lab.ps1 -Stage persistence</code>	delete the pod, prove the data survived
<code>.\run-lab.ps1 -Stage teardown</code>	kubect delete, then terraform destroy

Appendix C — References

- Oracle Cloud Infrastructure — Example Network Resource Configurations for OKE. docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengnetworkconfigexample.htm
- Oracle Cloud Infrastructure — Provisioning PVCs on Block Volume. docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengcreatingpersistentvolumeclaim_topic-Provisioning_PVCs_on_BV.htm
- Oracle Cloud Infrastructure — VCN Flow Logs. docs.oracle.com/en-us/iaas/Content/Logging/Task/vcn_flowlogs_eg.htm
- Terraform provider oracle/oci — containerengine_cluster, containerengine_node_pool, logging_log resource documentation.

- HashiCorp — Dynamic Blocks, Custom Conditions, Type Constraints, and Module Development documentation.

Appendix D — Console evidence: the network layer

Section 7 shows the cluster, the volume, the load balancer and the flow logs. This appendix shows the network those sit on: the VCN, the four subnets produced by four calls to one module, the three gateways, and the route tables and security lists each of those calls created. Everything below was built by Terraform; nothing in it was touched by hand.

The screenshot shows the 'Virtual Cloud Networks' page in the Oracle Cloud Console. The selected VCN is 'ejada-w3-dev-vcn', which is in an 'Available' state. The 'VCN Information' section displays the following details:

Property	Value
Compartment	intern-04-andrew-hany-cmp
Created	Aug 20, 2026, 00:21:51 UTC
IPv4 CIDR Block	10.0.0.0/16
IPv6 Prefix	—
Oracle-allocated IPv6 Prefix	—
BYOIP IPv6 Prefixes	—
ULA IPv6 Prefixes	—
OCID	ocid1.vcn.oc1.me-jeddah-1.amaaaaaavjakbnia3cc3nvvtui7vgdstpuvatuwqm5jmdwl3ed73yy0efekq

Figure 11. VCN *ejada-w3-dev-vcn*, 10.0.0.0/16, created 00:21:51 UTC — the first resource of the apply.

The screenshot shows the 'Subnets' page in the Oracle Cloud Console. It displays a list of four subnets created for the VCN 'ejada-w3-dev-vcn'. The subnets are:

Name	State	IPv4 CIDR Block	Oracle-allocated IPv6 Prefix	BYOIP IPv6 Prefixes	ULA IPv6 Prefixes	Subnet Access	Created
ejada-w3-dev-workers	Available	10.0.1.0/24	—	—	—	Private (Regional)	Aug 20, 2026, 00:21:55 UTC
ejada-w3-dev-pods	Available	10.0.32.0/19	—	—	—	Private (Regional)	Aug 20, 2026, 00:21:55 UTC
ejada-w3-dev-api	Available	10.0.0.0/29	—	—	—	Public (Regional)	Aug 20, 2026, 00:21:54 UTC
ejada-w3-dev-lb	Available	10.0.2.0/24	—	—	—	Public (Regional)	Aug 20, 2026, 00:21:54 UTC

Figure 12. The four subnets, from one module called four times. Sizes /29 to /19, two public and two private, all regional as OKE requires. The four were created within one second of each other.

The screenshot shows the 'Internet Gateways' page in the Oracle Cloud Console. It displays a list of one Internet Gateway created for the VCN 'ejada-w3-dev-vcn'. The gateway is:

Name	State	Route Table	Created
ejada-w3-dev-igw	Available	—	Aug 20, 2026, 00:21:52 UTC

Figure 13. Internet gateway — the default route for the two public subnets.

NAT Gateways

Search and Filter

Applied filters: Compartment `intern-04-andrew-hany-cmp`

Create NAT Gateway

Name	State	Public IP address	Route Table	Created
ejada-w3-dev-natgw	Available	80.225.65.33	—	Aug 20, 2026, 00:21:52 UTC

Page 1 of 1 (1 - 1 of 1 total items)

Items per page: 25

Figure 14. NAT gateway, public address 80.225.65.33 — outbound only, so private workers can pull images with no inbound path.

Service Gateways

Search and Filter

Applied filters: Compartment `intern-04-andrew-hany-cmp`

Create Service Gateway

Name	State	Services	Route Table	Created
ejada-w3-dev-sgw	Available	All JED Services In Oracle Services Network	—	Aug 20, 2026, 00:21:53 UTC

Page 1 of 1 (1 - 1 of 1 total items)

Items per page: 25

Figure 15. Service gateway to All JED Services In Oracle Services Network, resolved from the `oci_core_services` data source rather than hardcoded.

Route Tables

Search and Filter

Applied filters: Compartment `intern-04-andrew-hany-cmp`

Create Route Table

Name	State	Number of Rules	Created
ejada-w3-dev-workers-rt	Available	2	Aug 20, 2026, 00:21:54 UTC
ejada-w3-dev-pods-rt	Available	2	Aug 20, 2026, 00:21:54 UTC
ejada-w3-dev-lb-rt	Available	1	Aug 20, 2026, 00:21:53 UTC
ejada-w3-dev-api-rt	Available	1	Aug 20, 2026, 00:21:53 UTC
Default Route Table for ejada-w3-dev-vcn	Available	0	Aug 20, 2026, 00:21:51 UTC

Page 1 of 1 (1 - 5 of 5 total items)

Items per page: 25

Figure 16. Five route tables: one per subnet from the module, plus the VCN default. The default holds zero rules and nothing is attached to it — each subnet carries its own.

← ejada-w3-dev-vcn

ejada-w3-dev-workers-rt Available

Route Table

Details Route Rules Tags

Route Rules

Traffic within the VCN is handled by the VCN's local routing by default. Intra-VCN routing allows you more control over routing between subnets. [Learn more](#). If you're having problems, use [Network Path Analyzer](#) to check your connections.

Search and Filter Search

Add Route Rules Actions

☐	Destination ^	Target Type	Target	Route Type	Description	
☐	0.0.0.0/0	NAT Gateway	ejada-w3-dev-natgw	Static	Egress to the internet via NAT (no inbound)	...
☐	All JED Services in Oracle Services Network	Service Gateway	ejada-w3-dev-sgw	Static	Private path to the Oracle Services Network	...

Page 1 of 1 (1 - 2 of 2 total items) Items per page 25

of Use and Privacy Cookie Preferences Copyright © 2026, Oracle and/or its affiliates. All rights reserved.

Figure 17. The private route table: 0.0.0.0/0 via the NAT gateway, and the Oracle Services Network via the service gateway. Both rules and their descriptions come from `local.route_private`, rendered by the module's dynamic block.

Details IP administration Subnets Gateways Routing Private service access Security VLANs Work requests Tags

Security Lists

If you're having problems, use [Network Path Analyzer](#) to check your connections.

Search and Filter Search

Applied filters Compartment `intern-04-andrew-hany-cmp`

Create Security List

Name	State	Created	
ejada-w3-dev-workers-sl	Available	Aug 20, 2026, 00:21:53 UTC	...
ejada-w3-dev-pods-sl	Available	Aug 20, 2026, 00:21:53 UTC	...
ejada-w3-dev-lb-sl	Available	Aug 20, 2026, 00:21:53 UTC	...
ejada-w3-dev-api-sl	Available	Aug 20, 2026, 00:21:52 UTC	...
ejada-w3-dev-default-sl-empty	Available	Aug 20, 2026, 00:21:51 UTC	...

Page 1 of 1 (1 - 5 of 5 total items) Items per page 25

of Use and Privacy Cookie Preferences Copyright © 2026, Oracle and/or its affiliates. All rights reserved.

Figure 18. Five security lists: one per subnet, plus the VCN default adopted and emptied. `ejada-w3-dev-default-sl-empty` holds zero rules, closing the SSH-open-to-the-world list OCI creates with every VCN.

Details Security rules Tags

Ingress Rules

Search and Filter Search

Add Ingress Rules Actions

☐	Stateless	Source	IP Protocol	Source Port Range	Destination Port Range	Type and Code	Allows	
☐	No	10.0.2.0/24	TCP	All	10256		TCP traffic for ports: 10256	...
☐	No	10.0.1.0/24	All Protocols				All traffic for all ports	...
☐	No	0.0.0.0/0	ICMP			3, 4	ICMP traffic for: 3, 4 Destination Unreachable: Fragmentation Needed and Don't Fragment	...
☐	No	10.0.2.0/24	TCP	All	30000-32767		TCP traffic for ports: 30000-32767	...
☐	No	10.0.32.0/19	All Protocols				All traffic for all ports	...
☐	No	10.0.0.0/29	TCP	All	10250		TCP traffic for ports: 10250	...

Page 1 of 1 (1 - 7 of 7 total items) Items per page 25

of Use and Privacy Cookie Preferences Copyright © 2026, Oracle and/or its affiliates. All rights reserved.

Figure 19. Worker subnet, ingress — six rules: the load balancer on the NodePort range and on 10256, the control plane on 10250, pods and peer workers on all ports, and ICMP for path MTU discovery.

	Stateless	Destination	IP Protocol	Source Port Range	Destination Port Range	Type and Code	Allows
☐	No	10.0.0.0/29	TCP	All	6443		TCP traffic for ports: 6443
☐	No	All JED Services in Oracle Services Network	ICMP			3, 4	ICMP traffic for: 3, 4 Destination Unreachable: Fragment
☐	No	All JED Services in Oracle Services Network	TCP	All	All		TCP traffic for ports: All
☐	No	0.0.0.0/0	ICMP			3, 4	ICMP traffic for: 3, 4 Destination Unreachable: Fragment
☐	No	10.0.0.0/29	TCP	All	12250		TCP traffic for ports: 12250
☐	No	0.0.0.0/0	All Protocols				All traffic for all ports
☐	No	10.0.32.0/19	All Protocols				All traffic for all ports

Figure 20. Worker subnet, egress — seven rules, including 6443 and 12250 to the API endpoint and all TCP to the Oracle Services Network.

	Stateless	Source	IP Protocol	Source Port Range	Destination Port Range	Type and Code	Allows	Description
☐	No	10.0.1.0/24	All Protocols				All traffic for all ports	Workers -> pods
☐	No	10.0.0.0/29	All Protocols				All traffic for all ports	Control plane -> pods
☐	No	10.0.32.0/19	All Protocols				All traffic for all ports	Pod to pod

Figure 21. Pod subnet, ingress — three rules. The same module produced these three and the worker subnet's thirteen, with no branch written for either.

	Stateless	Source	IP Protocol	Source Port Range	Destination Port Range	Type and Code	Allows	Description
☐	No	0.0.0.0/0	TCP	All	80		TCP traffic for ports: 80	Public traffic to the application on port 80

	Stateless	Destination	IP Protocol	Source Port Range	Destination Port Range	Type and Code	Allows	Description
☐	No	10.0.1.0/24	TCP	All	10256		TCP traffic for ports: 10256	Load balancer -> kube-proxy health check
☐	No	10.0.1.0/24	TCP	All	30000-32767		TCP traffic for ports: 30000-32767	Load balancer -> NodePort range

Figure 22. *Load balancer subnet: one ingress rule on port 80 from the internet, generated by a for-expression over `var.app_ingress_ports`, and two egress rules to the workers.*